A Time-Based Sales Trend and Performance Analytics Dashboard for Retail Decision Support: A Case Study Using Python, Pandas, Plotly, and Streamlit

An Applied Data Analytics Case Study Based on the Open-Source "Afficionado Coffee Roasters" Dashboard Project

Department of Computer Science / Data Analytics
DEVANG, **Unified Mentor**

Abstract

Retail and food-service businesses generate continuous streams of transactional data, yet many small and mid-sized operators lack accessible tools to convert this data into operational insight. This paper presents an analysis of an open-source, web-based analytics dashboard built with Python, Pandas, Plotly, and Streamlit, developed to study time-based sales performance for a coffee retail business referred to as Afficionado Coffee Roasters. The dashboard ingests transactional CSV records, engineer's temporal features such as day-of-week and hour-of-day, and exposes interactive filters for store location, weekday, and time range. It produces revenue and quantity trend charts at daily, weekly, and monthly resolution, an hourly demand heatmap, and location-based comparison views. We describe the system architecture, the data processing pipeline, the visualization design choices, and the practical insights such dashboards can generate for staffing, inventory, and operational planning. We then discuss the broader pedagogical and methodological value of this class of project for data analytics curricula, situate it within related work on business intelligence and dashboarding tools, and identify its limitations and directions for future research, including statistical forecasting, anomaly detection, and multi-source data integration.

Keywords: sales analytics, time-series visualization, business intelligence, Streamlit, Plotly, Pandas, retail data, dashboard design

1. Introduction

Data-driven decision-making has become a baseline expectation for businesses of every size, yet the tools historically associated with business intelligence (BI) — enterprise platforms such as Tableau, Power BI, or Looker — often carry licensing costs, infrastructure requirements, or vendor lock-in that put them out of reach for small operators such as independent coffee shops, restaurants, and local retailers. At the same time, the open-source Python data science ecosystem has matured to the point where a single developer can build a fully interactive, browser-based analytics application using only freely available libraries. This paper examines one such application: a sales trend and time-based performance analytics dashboard developed for a coffee retail business, Afficionado Coffee Roasters, using Pandas for data manipulation, Plotly for interactive charting, and Streamlit for the web application layer.

The motivating problem is a common one in retail operations: transactional data — timestamped records of what was sold, when, and where — is routinely collected through point-of-sale systems but rarely analyzed beyond basic daily revenue totals. Important operational questions

go unanswered. Which hours of the day see peak demand, and is staffing aligned with that demand? Do weekday and weekend patterns differ enough to justify different inventory or staffing plans? Do different store locations behave similarly, or does one location's success not transfer to another? Answering these questions requires converting raw transaction timestamps into structured temporal features and visualizing the resulting patterns in a way that is both rigorous and accessible to non-technical stakeholders.

This paper is intended primarily as a teaching case study for university students studying data analytics, data visualization, or applied software engineering. It documents the system's design and data pipeline, articulates the analytical reasoning behind specific visualization choices, and discusses what such a project illustrates about the broader practice of applied data analytics — including its limitations, which are as instructive as its successes.

1.1 Objectives

This study has four objectives:

- Describe the architecture and data processing pipeline of a Python-based, time-aware sales analytics dashboard.
- Explain the temporal feature engineering and visualization techniques used to surface daily, weekly, hourly, and location-based sales patterns.
- Discuss the operational insights such a dashboard can support, including staffing optimization and revenue performance improvement.
- Evaluate the project's strengths and limitations as an example of applied analytics, and propose directions for extension.

2. Related Work and Background

Business intelligence dashboards have long been used to summarize organizational performance, traditionally through tools such as Microsoft Power BI, Tableau, and Excel-based pivot reporting. These platforms remain dominant in enterprise settings because of their mature data-connector ecosystems and polished reporting features, but they typically require proprietary licenses and are less amenable to custom, code-driven analysis. An alternative and increasingly common approach, particularly in academic and early-stage commercial settings, is to build lightweight web dashboards directly in Python. Frameworks such as Streamlit, Dash, and Panel allow a data scientist to turn a Pandas-based analysis script into an interactive web application without separate front-end development, lowering the barrier between exploratory analysis and a shareable decision-support tool.

Within the retail analytics literature and practitioner community, time-based decomposition of sales data — by hour, weekday, week, and month — is a well-established technique for identifying demand seasonality and operational bottlenecks, often visualized through heatmaps and grouped time-series charts. The coffee-retail and quick-service-restaurant domain in particular is well suited to this kind of analysis because demand is strongly periodic: morning and lunch rushes, weekday-versus-weekend differences, and location-specific foot traffic patterns are all directly observable in timestamped transaction data. The dashboard examined in this paper sits within this established practice, applying standard temporal feature engineering

and visualization techniques to a single-business dataset, and packaging the result as a publicly deployed, interactive web application rather than a static report.

3. System Overview

The dashboard is organized as a single-page Streamlit application (app.py) backed by a CSV dataset of point-of-sale transactions for Afficionado Coffee Roasters. The repository follows a conventional structure for a small Python data project, separating exploratory work from the deployed application:

Component	File / Folder	Purpose
Application entry point	app.py	Streamlit app: loads data, applies filters, renders charts
Raw and processed data	data/	Transaction-level CSV sales records
Exploratory analysis	notebook/	Jupyter notebooks for data cleaning, feature engineering, and chart prototyping prior to porting logic into the app
Dependency manifest	requirements.txt	Pinned Python packages (pandas, plotly, streamlit, etc.)
Documentation assets	images/	Dashboard screenshots for the project README

This separation of concerns — notebooks for exploration, a single application script for deployment — reflects a common and pedagogically useful workflow in applied data science: analytical logic is first developed and validated interactively, then refactored into a stable, reusable script suitable for a production-style web application. The finished application is deployed to Streamlit Community Cloud, making it accessible as a live web service without requiring end users to install Python or any dependencies locally.

3.1 Technology Stack

Layer	Technology and Role
Data processing	Pandas — loading, cleaning, and reshaping transaction data; computing groupby aggregations by date, weekday, hour, and location
Visualization	Plotly — interactive, hover-enabled line charts, bar charts, and heatmaps rendered directly in the browser
Application / UI layer	Streamlit — renders the page, manages interactive widgets (filters), and reactively re-runs the analysis when a user changes an input

Layer	Technology and Role
Data storage	CSV files — a lightweight, dependency-free format appropriate for a single-business dataset of this scale

4. Data Processing Pipeline

The analytical value of the dashboard depends almost entirely on the quality of its temporal feature engineering, since every chart in the application is a different view of the same underlying timestamp. The pipeline can be summarized in four stages.

4.1 Ingestion and Cleaning

Raw transaction records are loaded from CSV into a Pandas DataFrame. Typical cleaning steps for this kind of dataset include parsing transaction date and time fields into proper datetime objects, removing or imputing missing values, standardizing store-location labels, and verifying that revenue and quantity fields are numeric and non-negative. Correct datetime parsing is the single most consequential step in the pipeline, since an incorrectly parsed timestamp silently corrupts every downstream temporal aggregation.

4.2 Temporal Feature Engineering

From the cleaned datetime field, several derived features are engineered to support the dashboard's filters and aggregations:

- Date — used for daily and monthly trend lines.
- Day of week — used to compare weekday versus weekend demand and to build the weekday performance chart.
- Hour of day — used to build the hourly demand heatmap and to support the hour-range filter.
- Week / month period — used to roll daily transactions up into weekly and monthly trend aggregations.

These features transform a flat list of timestamped transactions into a structure that can be grouped along multiple, independent temporal dimensions, which is what allows the dashboard to answer different questions (“when during the day are we busiest?” versus “which months are strongest?”) from a single source table.

4.3 Aggregation

Pandas groupby operations aggregate the engineered features into the summary tables that feed each chart: revenue and transaction-quantity totals by date, by weekday, by hour, and by store location. Because Streamlit re-executes the script on every user interaction, these aggregations are recomputed reactively whenever a filter changes, which keeps every visualization synchronized with the user's current selection of location, weekday, and hour range.

4.4 Visualization

Plotly renders each aggregated table as an interactive chart embedded in the Streamlit page: line charts for daily, weekly, and monthly revenue trends; bar charts for day-of-week performance and location comparisons; and a two-dimensional heatmap (hour of day versus day of week, or hour versus date) for visualizing demand intensity. Because Plotly charts are interactive by default, users can hover over individual points to see exact values, zoom into a specific date range, and toggle series on or off — capabilities that a static image-based report cannot offer.

5. Key Features and Analytical Insights

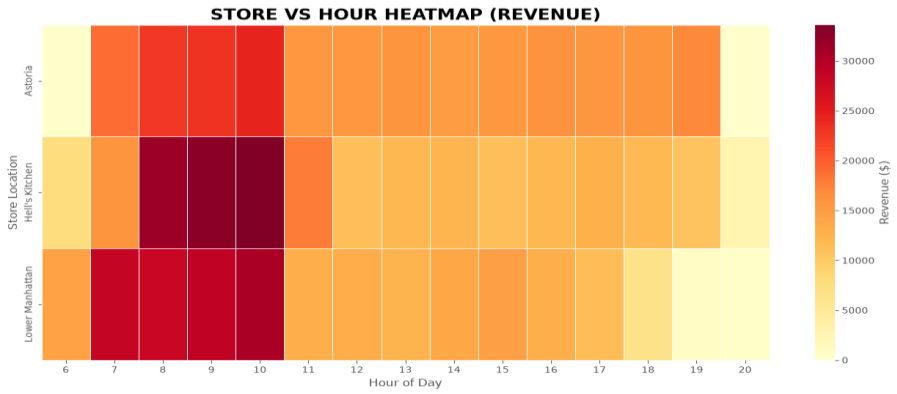
5.1 Interactive Filtering

Users can narrow the dataset by store location, day of week, and hour range. This allows a manager to, for example, isolate Saturday morning sales at a single location, directly answering a question that a static report would need a separate version to address.

5.2 Multi-Resolution Trend Analysis

Presenting trends at daily, weekly, and monthly resolution simultaneously serves different decision horizons: daily views expose short-term volatility and one-off events, weekly views smooth out day-to-day noise to reveal underlying momentum, and monthly views support longer-term planning and seasonal comparison. This is a standard but important practice in time-series business reporting, since a single resolution can either hide meaningful short-term variation or be overwhelmed by noise.

5.3 Hourly Demand Heatmap



The heatmap is arguably the dashboard's most operationally useful artifact, because it directly informs staffing decisions: a manager can identify the precise hours, on the precise days, where demand consistently peaks, and align staff schedules accordingly rather than relying on intuition. Heatmaps are particularly effective here because they compress two categorical dimensions (hour and weekday) and one continuous measure (transaction volume or revenue) into a single, immediately scannable visual.

5.4 Location-Based Comparison

Where the dataset spans multiple store locations, comparative charts allow stakeholders to see whether the patterns observed at one location generalize to others. This guards against the common analytical error of assuming that a single location's data represents the whole business, and can highlight location-specific factors — such as differing foot-traffic patterns by time of day — that should inform per-location, rather than company-wide, decisions.

6. Discussion

6.1 Educational and Practical Value

For university students studying data analytics, this project is a useful end-to-end case study precisely because it is small enough to fully understand yet complete enough to illustrate the full lifecycle of an applied analytics project: data ingestion, cleaning, feature engineering, aggregation, visualization design, and deployment as an interactive product. It also illustrates a workflow pattern that students should internalize — moving from exploratory notebook analysis to a structured, reusable application script — which mirrors how analytics work is typically operationalized in industry.

6.2 Limitations

- Descriptive, not predictive: the dashboard summarizes what already happened rather than forecasting future demand, which limits its use for proactive planning.
- Single dataset, single business: the underlying data and findings are specific to one coffee retailer and may not generalize to other retail contexts without re-validation.
- No statistical inference: differences observed between locations, weekdays, or hours are visual rather than tested for statistical significance, so apparent patterns could partly reflect random variation in smaller subsets of the filtered data.
- Static data source: the dashboard reads from a CSV snapshot rather than connecting to a live point-of-sale feed, so insights reflect a fixed historical period rather than real-time operations.
- Documentation gaps: at the time of this study, several README sections (research paper and demonstration video links, project evaluation score) were left as placeholders, indicating the project is best understood as an evolving student or portfolio project rather than a finished commercial product.

6.3 Directions for Future Work

- Incorporating time-series forecasting (e.g., seasonal decomposition or exponential smoothing) to project near-term demand rather than only describing historical demand.
- Adding statistical hypothesis testing to confirm whether observed differences between locations or time periods are significant.
- Connecting the dashboard to a live database or point-of-sale API for real-time monitoring rather than static CSV ingestion.

- Extending the location-comparison module with geospatial visualization if store coordinates are available.
- Adding automated alerts for anomalous sales drops or spikes, which would shift the tool from purely descriptive to proactively diagnostic.

7. Conclusion

This paper examined an open-source sales trend and time-based performance analytics dashboard built for a coffee retail business using Python, Pandas, Plotly, and Streamlit. By engineering temporal features from raw transaction timestamps and exposing them through interactive filters, multi-resolution trend charts, an hourly demand heatmap, and location comparisons, the dashboard converts routine point-of-sale data into actionable operational insight, particularly for staffing and demand planning. While the project is descriptive rather than predictive and is scoped to a single business's dataset, it serves as an instructive, complete example of an applied data analytics workflow, and provides a clear foundation for extension into forecasting, statistical testing, and real-time data integration in future work.